FATEC
São Caetano do Sul
.
Segurança da Informação

FDW - Ferramentas de Desenvolvimento para Web

Material de Apoio#1 JavaScript:Elementos Básicos

Professor: Veríssimo - carlos.pereira@fatec.sp.br

27/02/2026

Sobre este documento

Este documento objetiva deixar registrado o conteúdo abordado em sala de aula pelo professor. Importante destacar que a Nota de Aula serve como guia ao professor, bem como serve aos alunos como um norte, quanto ao conteúdo desenvolvido em sala de aula.
Este documento não tem a pretenção de ser uma única fonte para estudo. Para tal, o aluno deverá assistir às aulas e fazer uso (consulta) à bibliograa recomendada na ementa da disciplina, e à bibliograa complementar, apontada pelo professor.

Preâmbulo da Aula

Esta aula aborda coneitos fundamentais da linguagem JavaScript. Neste sentendido, trataremos dos seguintes tópicos:

- Entendendo JavaScript
- Manipulção de Variáveis (var, let, const)
- Controle de fluxo

Importante destacar que esta aula possui uma abordagem prática, na qual os elementos conceituai servem de guia para a parte prática da aula.

Contents

1 JavaScript- Introdução **1**

1.1 O que é JavaScript 1

1.2 Algumas Caracterísiticas Importantes do **JS** . . 2

1.3 Como podemos testar programas JavaScript? . . 3

1.3.1 Console do Navegador (Browser Console) 3

1.3.2 Editor de Código com Extensões de Execução de Código 4

1.3.3 Chamada por HTML 4

2 Utilizando JavaScript **7**

2.1 Ambiente Web - Font-End 7

2.2 Ambiente Servidor - Back-End 10

3 JavaScript - Manipulando Variáveis **11**

3.1 Tipos de Dados 11

3.2 Regras para os Identificadores 13

3.3 Bloco de Comando 13

3.4 Escopo de Variável 14

3.5 Identificador var 15

3.6 Identificadores let e const 15

4 Tratamento de Fluxo **17**

4.0.1 Estruturas Condicionais 17

4.0.2 Comando Swich 24

4.0.3 Estruturas while e do...while e for 26

5 Funções em JavaScript **29**

5.1 Introdução 29

5.2 Declarando Funções 29

6 Funções Anônimas **31**

6.0.1 Funções Anônimas - Sintaxe Básica . . . 32

6.0.2 Funções Anônimas: Passando um só parâmetro 33

Chapter 1

JavaScript- Introdução

1.1 O que é JavaScript

O JavaScript **(JS)** é uma linguagem de programação que permite que implementemos itens complexos em páginas Web. **(JS)** é geralmente utilizada em páginas Web, onde possui um papel importante para que uma página HTML possua elementos dinâmicos, pois, o HTML/CSS por si só, não possuem esta caracterísitica (Lembramdo que HTML é uma linguagem de marcação, e não uma linguagem procedural).

O poder da linguagem JavaScript transcende ao ambiente Web pois, com esta poderosa linguagem podemos desenvolver **App** Android/IoS, bem como também desenvolver servidores, **API-Rest** (Back-end).

1.2 Algumas Caracterísiticas Importantes do JS

Atenção a pontos importantes a considerar sobre a linguagem **(JS)**:

- É uma linguagem Interpretada (Não é compilada)
- É **Case Sentisitve**
- É uma linguagem **não-tipada**
- Suportando estilos de orientação a objetos, imperativos e declarativos (programação funcional)

1.3 Como podemos testar programas JavaScript?

O *JavaScript* pode ser executado tanto no lado do servidor, como no lado do cliente. Podemos seguir várias abordagens, dependendo do tipo de execução (cliente/servidor) que desejamos realizar. Nas subseções que seguem abordaremos a execução pelas seguintes abordagens:

- Navegador
- Editor VS Code
- Código por chamada em HTML

1.3.1 Console do Navegador (Browser Console)

Esta é uma maneira rápida e simples de testar pequenos trechos de código *JavaScript*.

1. Abra o console do navegador pressionando **F12** ou clicando com o botão direito do mouse na página e selecionando "Inspecionar" ou "Console".
2. Localize a aba "***console***"
3. Podemos inserir código JavaScript **diretamente** no console e pressionar Enter para ver os resultados.

O resultado podemos ver na figura 1.1 , onde podemos observar qur a mensagem está em destaque.

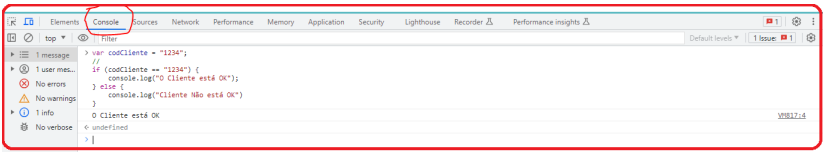

Figure 1.1: Utilizando Browser para executar JS

1.3.2 Editor de Código com Extensões de Execução de Código

Muitos editores de código, como **Visual Studio Code**, oferecem extensões que permitem executar código *JavaScript* diretamente. Podemos escrever o código em um arquivo *.js* e usar essas extensões para executá-lo no ambiente de desenvolvimento, conforme podemos observar àa figura 1.2, onde utilizamos o comando *node*: **node teste.js**

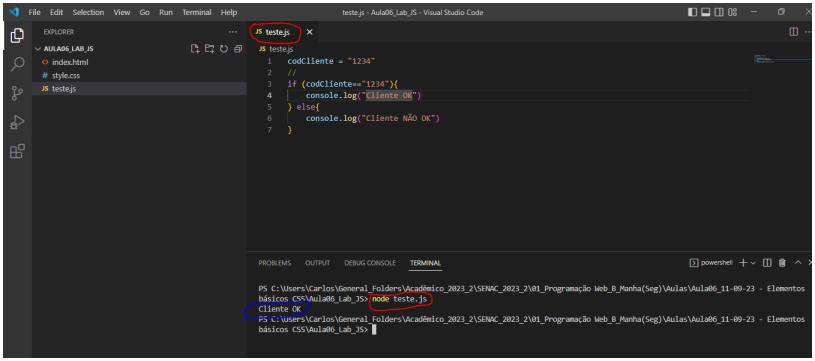

Figure 1.2: Executando JS com *Node*

1.3.3 Chamada por HTML

Para chamadas de script JavaScript em códigos HTML, podemos fazê-lo de duas formas, conforme seguem às subseções abaixo.

Chamada incorporada no código HTML

Podemos criar scripts no código HTML ou fazer chamadas de arquivos .js para executar códigos, a partir de páginas HTML, conforme podemos observar à figura 1.3, onde utilizamos o chamamos o script, a partir de um evento de botão.

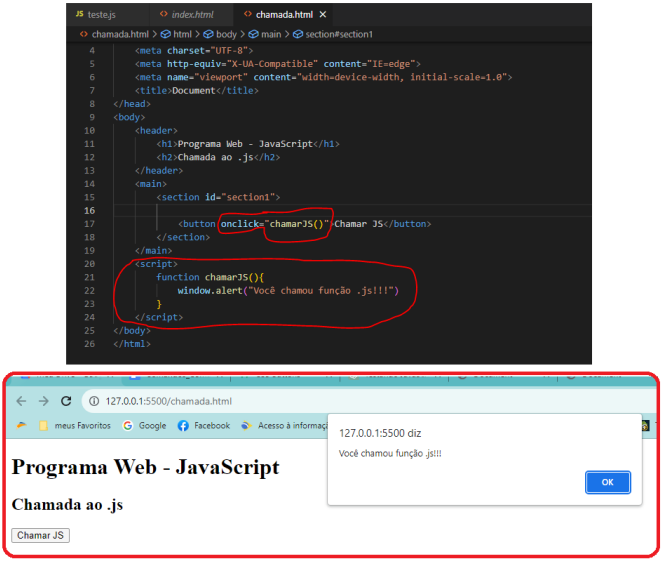

Figure 1.3: Executando JS chamada no HTML

Chamada à arquivos externos ao código HTML

Podemos executar scripts Javascript, a partir de arquivos externos ao código HTML. Esta é a forma mais recomedada, conforme abordado à aula anterior (que versou sobre HTML). Podemos observar à figura 1.4, onde chamamos uma function,partir de um evento de botão, que se encontra em um arquivo .js.

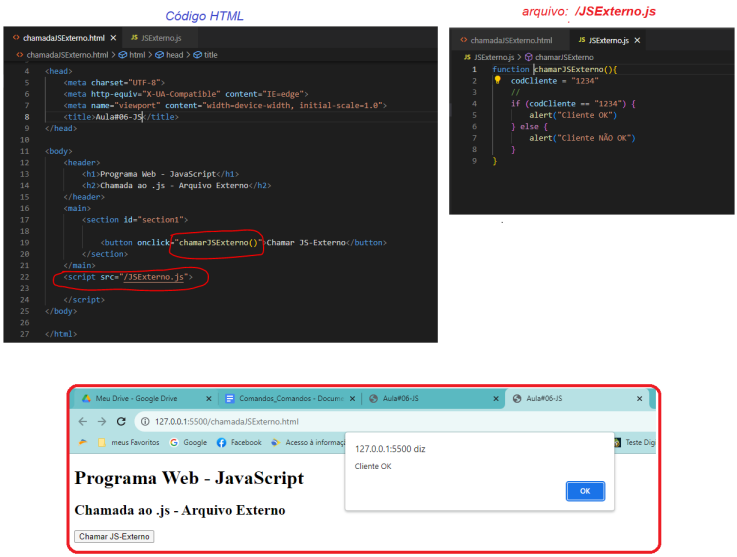

Figure 1.4: Executando JS por chamada à arquivo externo

Chapter 2

Utilizando JavaScript

O **JS** é uma linguagem de programação que permite implementar funcionalidades mais complexas em páginas web **(Front-end)**, bem com em ambientes de servidores **(Back-end)**

2.1 Ambiente Web - Font-End

O trecho de código 6.1 abaixo, ilustra a utilização da linguagem **JS** em uma página **HTML**. Observe que a página HTML aciona a função **clicou()**, a partir do envento **onclick** - **"Button"**

```

/*-----------------------------------------------*
  * Fatec - Ferramentas de Desenvolvimento para Web   *
  *            Programacao Web                        *
  *  Prof. Verissimo                    Set/2023     *
  *-----------------------------------------------*
*/
<!DOCTYPE html>
<html lang="en">

<head>
```

```
12      <meta charset="UTF-8">
13      <meta http-equiv="X-UA-Compatible" content="IE
           =edge">
14      <meta name="viewport" content="width=device-
           width, initial-scale=1.0">
15      <title>PW-JavaScript-Introducao</title>
16      <link rel="stylesheet" href="style.css">
17  </head>
18
19  <body>
20      <header>
21          <h1>Programa Web - JavaScript</h1>
22          <h2>Interacao com Usuario</h2>
23      </header>
24      <main>
25          <section id="section1">
26              <h3>Interacaocom o Botao</h3>
27              <button onclick="clicou()">Clicar Aqui
                   </button>
28          </section>
29      </main>
30      <script>
31          function clicou(){
32              window.alert("Voce CLICOU no Botao!!!"
                   )
33          }
34      </script>
35  </body>
36
37  </html>
```

Listing 2.1: Exemplo inteção com JS

O resultado podemos ver na figura 2.1 , onde podemos observar qur a mensagem está em destaque.

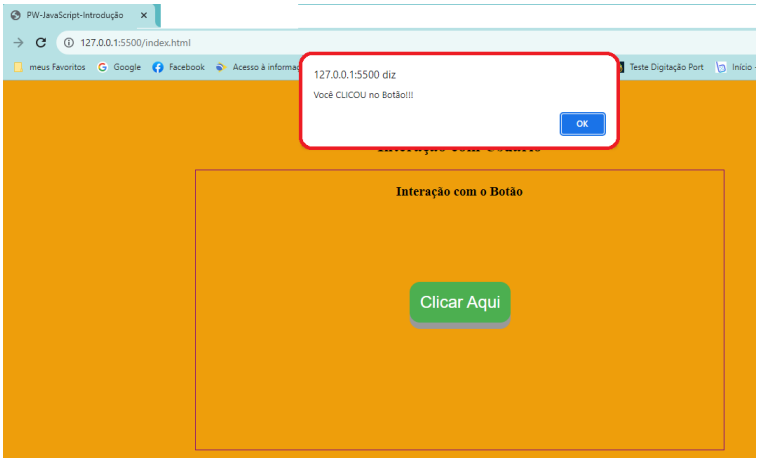

Figure 2.1: Resultado do exemplo JS acionado à página HTML

2.2 Ambiente Servidor - Back-End

A figura 2.2 ilustra a utilização da linguagem **JS** em uma Aplicação no Servidor **API**. Neste exemplo a API responde a uma requisição: Devolve em formato de protocolo **JSON**

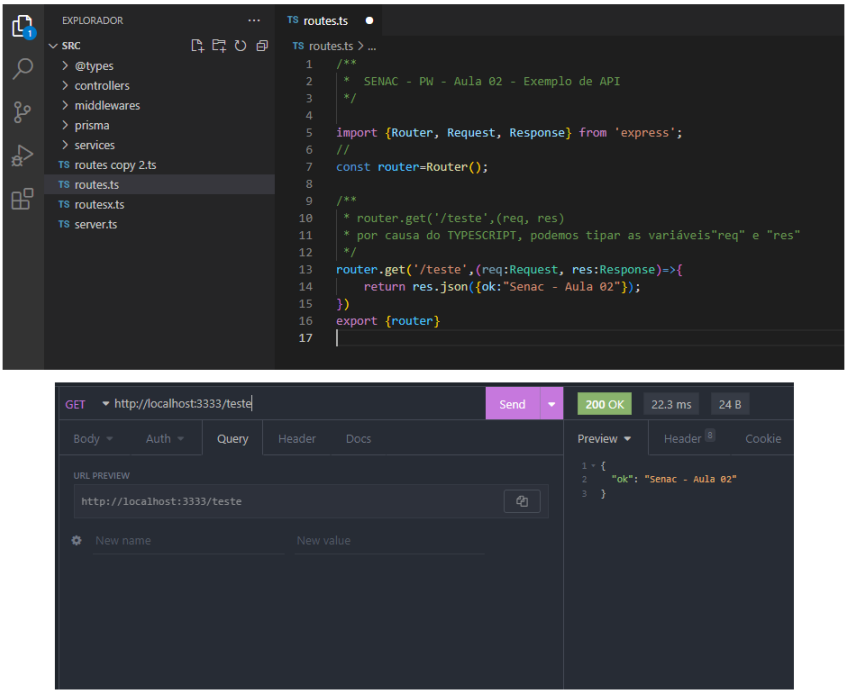

Figure 2.2: Exemplo JS no Servidor - API

Chapter 3

JavaScript - Manipulando Variáveis

3.1 Tipos de Dados

- Boolean: entidade lógica e pode ter dois valores: verdadeiro(true) ou falso(false).
- Null:tem exatamente um valor: null
- Undefined:Uma variável que não foi atribuída a um valor específico
- Number:O tipo number possui apenas um inteiro que tem duas representações: 0 é representado como -0 ou +0.
- BigInt:é um tipo de dado numérico que representa inteiros no formato de precisão arbritrária.
- String:O tipo String em JavaScript é usado para representar dados textuais. Isto é um conjunto de "elementos" de valores de 16-bits

- Symbol: Um Symbol é um valor primitivo único e imutável e pode ser usado como chave de uma propriedade[1] de Object
- **Object**: refere-se a uma estrutura de dados contendo dados e instruções para se trabalhar com estes dados.

O operador **typeof** retorna uma string indicando o tipo de um operando.
O operador typeof pode ser utilizado das seguintes maneiras:

- **typeof** operando

Exemplo:

- console.log(typeof "3.14");

Resultado: **string**

[1]Uma propriedade Javascript é uma característica de um objeto, frequentemente descrita como atributos associados à uma estrutura de dados.

3.2 Regras para os Identificadores

Os nomes de variáveis [2], chamamos de **"Identificadores"**. Temos que atentar para as seguintes regras:

- Podemos começar com letra, $ ou _ (Underline)
- Não pode começar com **números**
- podem conter letras e números
- é possível usar acentos e símbolos
- Não podem conter espaços
- não podem ser palavras reservadas

3.3 Bloco de Comando

Em JavaScript, um bloco de comando é um conjunto de instruções delimitado por chaves **{ }**. O bloco de comando é usado para agrupar uma ou mais instruções e é frequentemente usado em estruturas de controle de fluxo, como if/else, while, for, switch, try/catch, entre outros. O trecho de código **??** abaixo, demonstra blocos de comando em uma estrutura de desição:

[2] Na W3schools usamos camelCase para nomes de identificadores (variáveis e funções).

```
/**-------------------------------------------*
 * Fatec - Ferramentas de Desenvolvimento para Web
    *
 * Exemplo de bloco de comando                *
 *-------------------------------------------*/
if (true) {
  let x = 10; // a vari vel x    declarada dentro
      do bloco if
    console.log(x); // a vari vel x pode ser
        acessada dentro do bloco if
  }

  //cls
  console.log("Valor de x",x); // a vari vel x
      n o pode ser acessada fora do bloco if,
     resultando em um erro
```

Listing 3.1: Exemplo Bloco de Comando

3.4 Escopo de Variável

Escopo é a acessibilidade de objetos, variáveis e funções em diferentes partes do código, isto significa dizer que, escopo é o que determina quais são os dados que podem ser acessados em uma determinada parte do código . As palavras chaves **let** e **const** permitem que trabalhemos não só com o escopo de funções (**var**), mas também com o escopo dos blocos.

Ao utilizamos JS é imprescindível que entendamos como aplicar escopo de variável:[3] de forma correta.

- Escopo global
  - É definida quando declaramos uma variável fora de qualquer função - ela torna acessível a qualquer

[3]Escopo de variável é o local de nosso código onde uma determinada variável pode ser acessada:global ou local

parte da nossa aplicação ou site, podendo ser lida e alterada

- Escopo Local
  - É declarada dentro de uma função

Escopos criados por funções são chamados de **function scopes**, enquanto escopos criados por estruturas de controle são chamados de **block scopes**.

3.5 Identificador var

O identificador **var** age sobre o **escopo da função**:Ao declaramos uma variável sem o uso da palavra reservada **var** estaremos criamos uma variável **global implicitamente**, e automaticamente ela se torna global independente de onde ela for definida.

Uma variável global é definida quando declaramos uma variável fora de qualquer função, portanto, estão acessíveis em qualquer lugar em nosso código. O escopo de uma variável declarada com ***var*** é seu contexto atual em execução, o qual é a função a qual pertence ou, para variáveis declaradas fora de qualquer função, o escopo é o global.

3.6 Identificadores let e const

A grande mudança trazida pelo **ES2015** foram a introdução de **let** e **const** como maneiras de definirmos variáveis.
Essas **keywords** permitem que trabalhemos não só com o escopo de funções, mas também com o escopo dos blocos

- let tem escopo de bloco
- let pode ser atualizado, mas não declarado novamente.

- Declarações com const têm escopo de bloco
- Variáveis declaradas com const mantêm valores constantes
- const não pode ser atualizado nem declarado novamente
- Cada declaração com const deve ser inicializada no momento da declaração.
- Declarações de const somente podem ser acessadas dentro do bloco onde foram declaradas.

Chapter 4

Tratamento de Fluxo

4.0.1 Estruturas Condicionais

As declarações condicionais, em qualquer linguagem de programação, nos permitem representar tomadas de decisão,a partir da escolha que deve ser feita.

Abordaremos aqui as seguintes estruturas condicionais:

- Estrutura de Decisão simples:*if*
- Estrutura de Decisão *if*...*else*
- Encadeamento de decisões (*if*...*else* **encadeado**)

Estrutura de Decisão *if*

Esta estrutura permite avaliar uma condição e, a partir dela, executar um bloco de código, somente se o resultado for Verdadeiro.

Conforme demonstrado à figura 4.1 podemos verificar que esta estrutura permite a execução de um bloco de código, de

acordo com o resultado da análise da condição (Comando *if*)

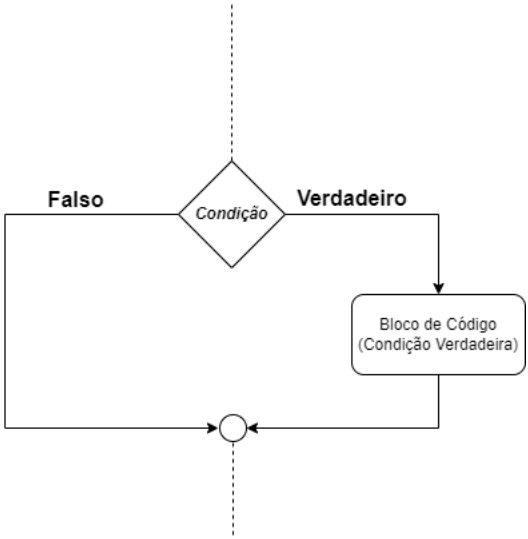

Figure 4.1: Estrutura de decissão if

A **sintaxe** para declarar um comando *if* *é:*

```
/**-------------------------------------------*
 *       Fatec - Ferramentas de Desenvolvimento
    para Web   *
 *              Sinstaxe comando if            *
 *-------------------------------------------*/
 if (condicao){
  //bloco de codigo: Condicao Verdadeira
}
```

Listing 4.1: Sintaxe da estrutura if simples

Comando *if...else*

Esta estrutura condicional permite avaliar uma condição e, a partir dela, executar diferentes linhas de código.

Conforme demonstrado à figura 4.2 podemos verificar que esta estrutura permite a execução de um bloco de código verdadeiro ou um bloco de código falso, de acordo com o resultado da análise da condição (Comando if)

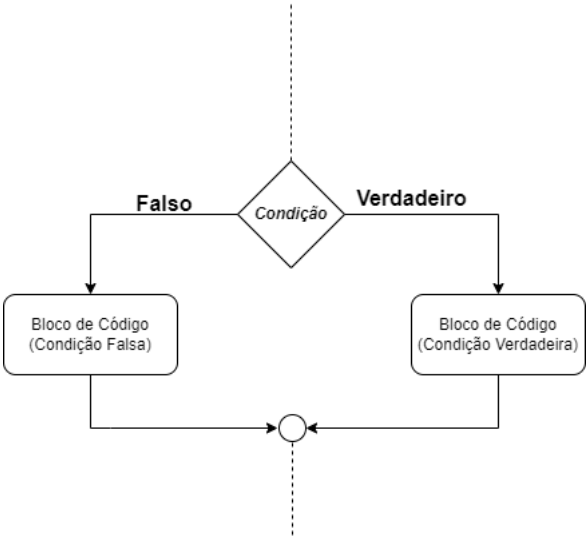

Figure 4.2: Estrutura de decissão if...else

*A **sintaxe** para declarar um comando if...else é:*

```
/**-------------------------------------------*
 *      Fatec - Ferramentas de Desenvolvimento para
     Web        *
 *         Sinstaxe comando if...else          *
 *-------------------------------------------*/
 if (condicao){
  //bloco de codigo: Condicao Verdadeira
} else {
  //bloco de codigo: Condicao Falsa
}
```

Listing 4.2: Sintaxe da estrutura if...else

Comando *if*...*else* encadeados

Esta estrutura condicional valiar permite sucessivas condições e, a partir dela, executar diferentes blocos de código.

Conforme demonstrado à figura 4.3 podemos verificar que esta estrutura permite a execução de um bloco de código verdadeiro ou um bloco de código falso, de acordo com o resultado da análise da condição (Comando if)

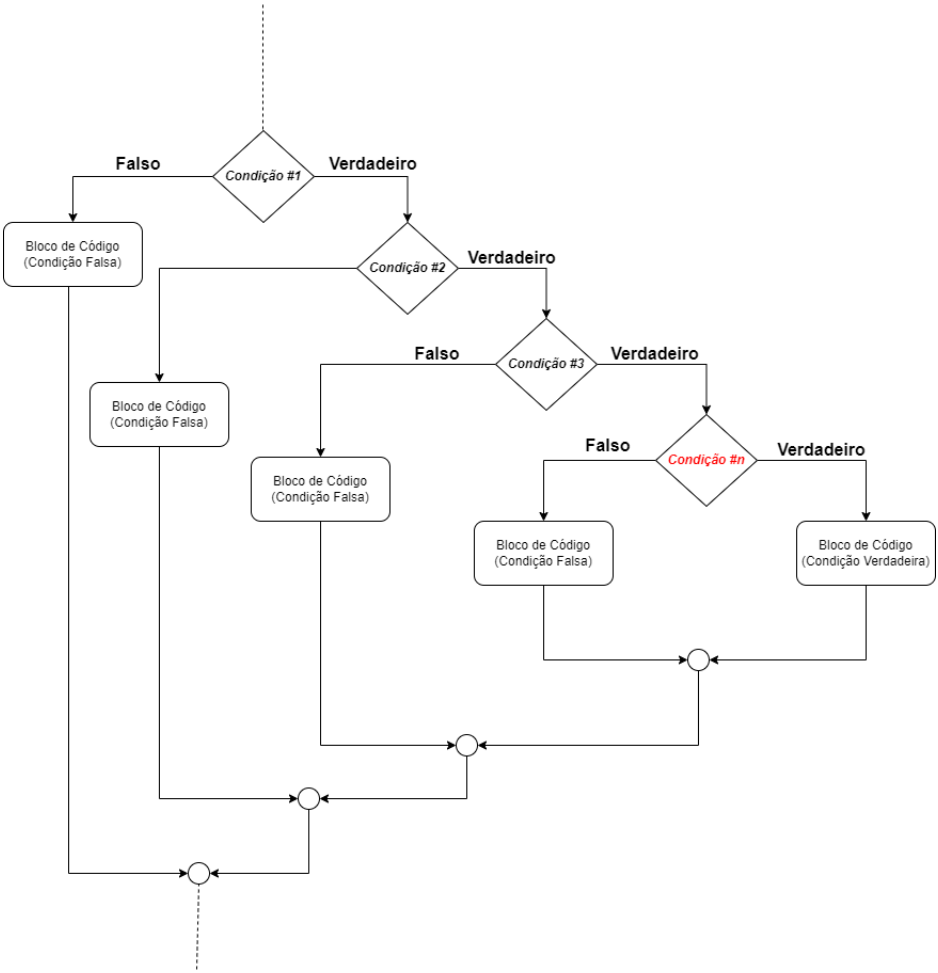

Figure 4.3: Estrutura de if...else encadeados

*A **sintaxe** para declarar if...else encadeados é:*

```
/**-------------------------------------------*
 *       Fatec - Ferramentas de Desenvolvimento
    para Web      *
 *     Sinstaxe comando if...else encadeados   *
 *-------------------------------------------*/
 if (condicao#1){
     if (condicao#2) {
        if (condicao#3) {
            if (condicao#n) {
                    // bloco#n de codigo: Condicao
                       verdadeira
            } else {
                   //bloco#n de codigo: Condicao
                      Falsa
            }
        } else {
            //bloco#3 de codigo: Condicao Falsa
        }
     } else {
        //bloco#2 de codigo: Condicao Falsa
     }
 } else {
     //bloco#1 de codigo: Condicao Falsa
 }
```

Listing 4.3: Sintaxe da estrutura if...else encadeado

Operadores Condicionais

O quadro 4.1 mostra os operadores de comparação na linguagem javScript.

Table 4.1: OPeradores de comparação JavaScript

Operador	*Operação*	*Exemplo*
$>$	Maior que	$(a > b)$
$<$	Menor que	$(a < b)$
$>=$	Maior ou igual a	$(a >= b)$
$<=$	Menor ou igual a	$(a <= b)$
$==$	Igual a	$(a == b)$
$! =$	Diferente a	$(a! = b)$
$===$	Idêntico a	$(a === b)$
$! ==$	Não Idêntico a	$(a! == b)$
$\&\&$	E/AND	$(a\&\&b)$
$\|\|$	OU/OR	$(a\|\|b)$

4.0.2 Comando Swich

*A instrução switch é usada para executar diferentes ações com base em diferentes condições. Este comando é uma alternativa ao comando **if**.*

A sintax para declaração de switch é:

```
1  switch(expression) {
2    case x:
3      // code block
4      break;
5    case y:
6      // code block
7      break;
8    default:
9      // code block
10 }
```

Listing 4.4: Sinxtax para declarar Swich

A listagem 4.5 abaixo, podemos ver as a utilização do switch.

Observer os seguintes pontos:

- ***linha 8** inicia-se o comando, utilizando a variável diaObtido*
- *O comando break dever ser colocado dentro de cada case obtido*
- *à linha 30, o comando **default** serve para identificar que os testes feitos em todos comandos case, não foram atendidos.*

```
/**-----------------------------------------*
 *      Fatec - Ferramentas de Desenvolvimento para
     Web       *
 *                Exemplo de Switch           *
 *-----------------------------------------*/

let diaObtido=new Date().getDay();
diaObtido=5;
switch (diaObtido) {
    case 0:
      diaDaSemana = "Domingo";
      break;
    case 1:
        diaDaSemana = "Segunda-Feira";
      break;
    case 2:
        diaDaSemana = "Terca-Feira";
      break;
    case 3:
        diaDaSemana = "Quarta-Feira";
      break;
    case 4:
        diaDaSemana = "Quinta-Feira";
      break;
    case 5:
        diaDaSemana = "Sexta-Feira";
      break;
    case 6:
        diaDaSemana = "Sabado";
        break;
    default:
    diaDaSemana = "Dia Invalido";
    break;
  }
  console.log("Hoje eh:"+diaDaSemana)
```

Listing 4.5: Exemplo manipulacao de Switch

4.0.3 Estruturas while e do...while e for

Dos paradigmas da lógica de programa, a repetição de um trecho de código, é uma grande ferramenta para os programadores. As estruturas de repetição while; do...while e for proporcionam implementar este paradigma.

Para quem quer ter sucesso em qualquer linguagem de programação, é imperativo ter pleno domínio, destas estruturas de repeção.

Repetição while

A **sintaxe** para declarar *while* é:

```
/**-----------------------------------------*
 *      Fatec - Ferramentas de Desenvolvimento
    para Web      *
 *       Sinstaxe da estrutura while      *
 *-----------------------------------------*/
 while (condicao) {
    //Bloco de comandos
 }
```

Listing 4.6: Sintaxe da estrutura while

Repetição do...while

A **sintaxe** para declarar *do...while* é:

```
/**-----------------------------------------*
 *      Fatec - Ferramentas de Desenvolvimento
    para Web      *
 *     Sinstaxe da estrutura do...while      *
 *-----------------------------------------*/
 do {
    //Bloco de comandos
    }
    while (condicao)
```

Listing 4.7: Sintaxe da estrutura do...while

for

A **sintaxe** para declarar *for* é:

```
/**-----------------------------------------*
 *     Fatec - Ferramentas de Desenvolvimento para
    Web        *
 *          Sinstaxe da estrutura for          *
 *-----------------------------------------*/
 for (inicializacao; condicao; incremento) {
    //Bloco de comandos
    }
```

Listing 4.8: Sintaxe da estrutura for

Chapter 5

Funções em JavaScript

5.1 Introdução

Funções possuem um papel muito importante na programação estrutural, e adicionalmente, ajudam muito na modularização no programa. Neste caso, estamos a falar de divisão do programa em partes menores e logicamente relacionadas. Existem diversas maneiras de se declarar uma função.

Podemos entender que Funções são blocos de construção fundamentais em JavaScript. Uma função é um procedimento de JavaScript — um conjunto de instruções que executa uma tarefa ou calcula um valor.

5.2 Declarando Funções

A forma mais comum de se declarar uma função é através do uso da palavra chave **function** de maneira similar a como elas são declaradas na linguagem C Cabe ressaltar que JavaScript **não definimos o tipo de retorno** e **nem o tipo dos argumentos**.

A figura 5.1-A, mostra a sintaxe da declaração de uma função , sem a passagem de parâmetros. Já a figura 5.1-B mostra a sintaxe da declaração de uma função, com passagem de parâtetros.

fig/fig01_funcao.PNG

Figure 5.1: Sintaxe da declaração da *function*

Funções podem receber parâmetros, assim como podem retornar valores, ao seus "chamadores". A listagem 6.1 mostra um exemplo decleração e utilização de uma função chamada ***soma***, que retorna o resultado da soma de dois valores.

```
/**-------------------------------------------*
 * SENAC - TADS - Programacao Web              *
 * Exemplo de Funcao                           *
 *-------------------------------------------*/
// Aqui , declaramos a funcao
function somar(parcela1, parcela2) {
    let resultado = parcela1 + parcela2;
    return resultado;
}
let valor1 = 3;
let valor2 = 6
somaDosValores = somar(valor1, valor2);
// vamos chamar a funcao e mostrar o resultado
console.log(`O resultado da soma de ${ valor1 } +
   ${valor2} = ${ somaDosValores } `);
```

Listing 5.1: Exemplo de *function*

Chapter 6

Funções Anônimas

Vimos à aula #2; que funçoes são *"segmentos de códigos"*, que podem ser utilizados pelo programa, ou por outros programas.
Funções podem receber parâmetros, assim como podem retornar valores, ao seus ***"chamadores"***.A listagem 6.1 mostra um exemplo decleração e utilização de uma função chamada *soma*, que retorna o resultado da soma de dois valores.

```
/**-------------------------------------------*
 *       SENAC - TADS - Programacao Web        *
 *           Exemplo de Funcao                 *
 *-------------------------------------------*/
 //Aqui, declaramos a funcao
function somar(parcela1,parcela2){
    let resultado = parcela1+parcela2;
    return resultado;
}
let valor1=3;
let valor2=6
somaDosValores = somar(valor1, valor2);
//vamos chamar a funcao e mostrar o resultado
console.log(`O resultado da soma de ${valor1} + ${
   valor2} =${somaDosValores} `);
```

Listing 6.1: Exemplo de Função

Iremos ver nesta seção uma outra forma de criação de funções: **Funções Anônimas**, também conhecidas como

"Arrow Function"

6.0.1 Funções Anônimas - Sintaxe Básica

Função anônima, ou *Arrow Function* é uma função que não possui nenhum nome associado a ela.Neste caso usamos apenas a palavra-chave ***function*** sem o nome da função. A sintaxe para a declaração de uma função anônima é:

() ⇒{}

Onde:

- () = Lista de parâmetros recebidos pela função
- ⇒ valor fixo, Indicar que se trata de uma"Arrow" function
- {} = Bloco de código que representa o ***corpo da função***

Veja exemplo abaixo, a listagem 6.2, que à exemplo da figura 6.1, efetua a soma entre dois números, porém agora está no formato de "Arrow Function".

Observe que à **linha 8** atribuimos à **variável** *somaDosValores* o resultado do processamento da **Arrow Function** (Lembre-se que **Arrow Function** começa na lista de parâmetros (neste caso *parcela1* e *parcela2*), depois temos o símbolo seta, seguida do corpo da função dentro abre/fecha chaves ⇒**{}**.

```
/**-------------------------------------------*
 *      SENAC - TADS - Programacao  Web        *
 * Exemplo de Funcao Anonima (Arrow Function) *
 *-------------------------------------------*/
let valor1=3;
let valor2=6
// Aqui estamos a definir a Arrow Function
let somaDosValores = (parcela1, parcela2) => {
    let resultado = parcela1+parcela2;
    return resultado;
};
// vamos chamar a funcao e mostrar o resultado
console.log(`O resultado da soma de ${valor1} + ${valor2} =${somaDosValores(valor1,valor2)} `);
```

Listing 6.2: Exemplo de Função Arrow Function

6.0.2 Funções Anônimas: Passando um só parâmetro

Ao passar **somente um parâmetro**, parênteses são opcionais, conforme demonstrado na listagem 6.3.

Observe que à **linha 8** atribuimos à **variável** *nmCliente* o resultado do processamento da **Arrow Function** (Lembre-se que **Arrow Function** começa na lista de parâmetros (neste caso *parmNomeCliente*), depois temos o símbolo seta, seguida do corpo da função dentro abre/fecha chaves ⇒**{}**.

```
1  /**-------------------------------------------*
2   *      SENAC - TADS - Programacao Web         *
3   *  Exemplo de Funcao anomina com um parametro*
4   *-------------------------------------------*/
5
6  nomeCliente="Joaquim Jos  da Silva Xavier";
7  //Defincao de funcao Anonima passando somente um parametro
8  let nmCliente = parmNomeCliente => {
9      console.log("Bem vindo Sr."+parmNomeCliente)
10 }
```

```
11 nmCliente(nomeCliente);
```

Listing 6.3: Exemplo de Função Anômina com um parâmetro

Observe aqui à **linha 11** que estamos a chamar a função anônima, passando um único valor como parâmetro (No caso o conteúdo da variável *nomeCliente*)